Part 7: Parallelization with Agents

A Comprehensive Guide to Agentic Coding Tools

Graduate Research Seminar

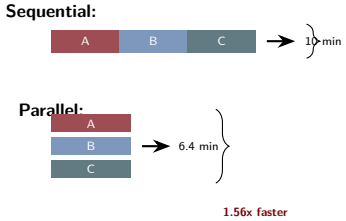
University of South Carolina

Spring 2026

Outline

Why Parallelization Matters in Agentic Coding

- **36% Performance Improvement**
  - Sequential: 10 minutes
  - Parallel: 6.4 minutes
- **90% Quality Improvement**
  - Multiple agents cross-verify work
  - Catches more errors
- **Real-world Impact**
  - 1 hour analysis $\rightarrow$ 17 minutes
- **Scalability**
  - More tasks without proportional time



Performance Gains Across Task Types

Research Phase Tasks

40–50% faster

- Literature review + codebase exploration
- 8 min instead of 15 min

Data Processing Tasks

60–70% faster

- Multiple parameter sweeps
- 5 min instead of 45 min

Code Analysis Tasks

35–45% faster

- Trace flow + search patterns + check docs
- 12 min vs 20 min sequential

Build & Test

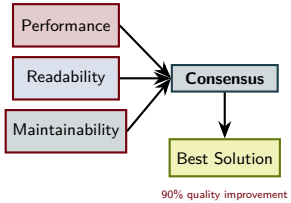
30–40% faster

- Lint + test + docs simultaneously
- 3 min instead of 5 min

How Parallelization Improves Code Quality

Multiple Perspectives

- Agent 1: Performance focus
- Agent 2: Readability focus
- Agent 3: Maintainability focus
- Consensus emerges on best solution



Cross-Validation

- 2/3 agents agree $\rightarrow$ likely correct
- Divergent results $\rightarrow$ investigate
- Reduces hallucination through consensus

Coverage Improvement

- Agent A: Functionality tests
- Agent B: Edge case tests

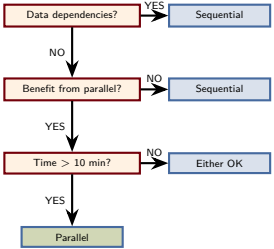
Decision Matrix: Parallel vs Sequential

Use Parallel When:

- Tasks have NO data dependencies
- Sequential time $>$ 10 minutes
- Multiple perspectives improve quality
- Budget for increased tokens
- Minimal synchronization points

Use Sequential When:

- Task B requires output from Task A
- Sequential time $<$ 5 minutes
- Token budget is constrained
- Tasks are tightly coupled



Understanding the Token Cost of Parallelization

Token Multiplication Factor

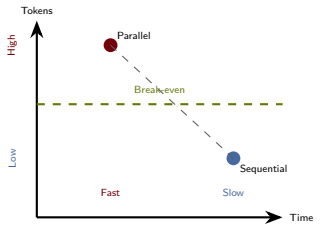
- 1 agent: 10,000 tokens
- 3 parallel: ∼30,000 tokens
- 5 parallel: ∼50,000 tokens

Why Tokens Multiply

- Each agent receives full context
- Independent reasoning (not shared)
- System prompts repeated

Mitigation Strategies

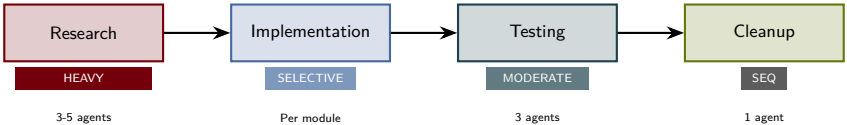
- Use smaller models for parallel tasks
- Compress context for parallel agents
- Parallelize only critical path



Cost-Benefit Rule

Worth it if: Time value > Token cost

Parallelization in Your Development Process



Research Phase

Explore codebase + search literature + identify patterns (all simultaneously)

Review/Testing Phase

Different test suites in parallel; different linting rules simultaneously

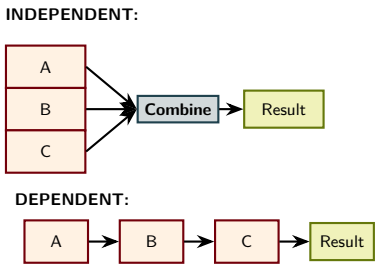
The Foundation: Task Independence

Independent Tasks

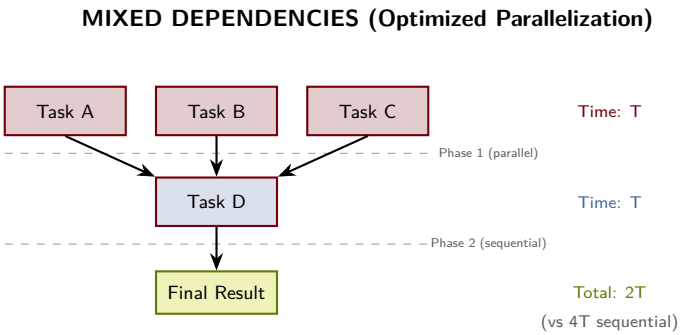
- Can run in any order, produce same result
- Agent A: Analyze performance
- Agent B: Check style compliance
- Agent C: Review security
- Order ABC, CAB, BCA $\rightarrow$ same result

Dependent Tasks

- B requires A's output
- Agent A: Parse data $\rightarrow$ `parsed_data`
- Agent B: Analyze `parsed_data`
- Must run A first, then B



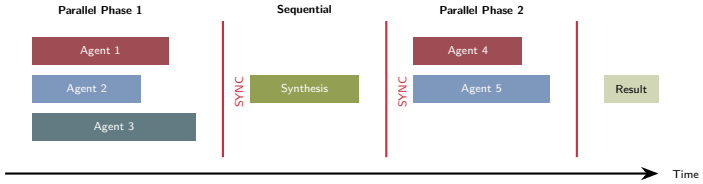
Dependency Graph Visualization



Key Insight

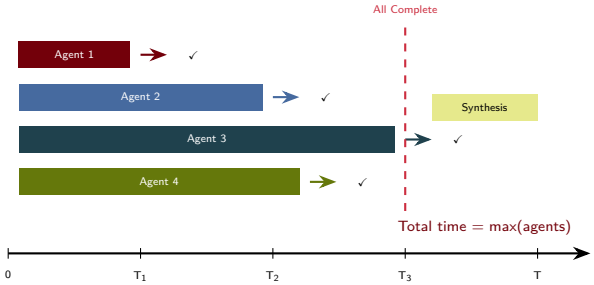
You can parallelize tasks at the same "level" (with no edges between them). This often requires careful problem decomposition.

Synchronization Points



Full Barrier	Partial Barrier	Timeout Barrier
Wait for ALL agents before proceeding	Wait for ANY N agents (redundancy)	Wait up to 5 min, proceed with results

Batch Execution and Agent Lifecycle



Lifecycle Characteristics

- All agents receive FULL context
- Agents don't communicate with each other
- Each agent has isolated workspace
- Results collected after completion

Maximum Parallelism

- Claude Code: 10 concurrent agents
- Cursor: 8 concurrent agents
- Continue CLI: Unlimited (rate limited)

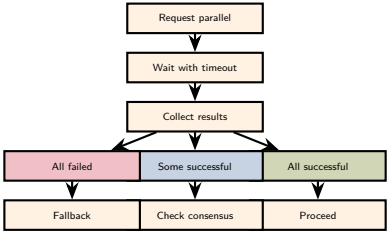
Error Handling and Fault Tolerance

Failure Modes

- Agent timeout (30-60 min limit)
- Agent crash (OOM)
- Invalid output (malformed)
- Partial failure

Recovery Strategies

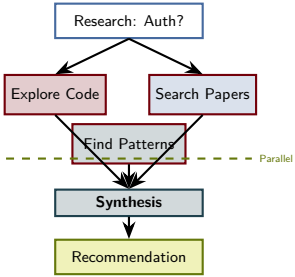
- **Consensus**: 2/3 succeed → use majority
- **Redundancy**: Run 5 agents, need 4+ agree
- **Fallback**: Sequential retry if parallel fails



Pattern 1: Research & Literature Review

Three-Agent Research Breakdown

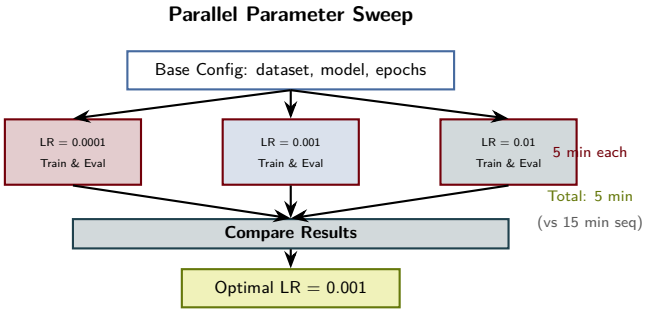
- **Agent 1 - Codebase Explorer**
  - File organization, module boundaries
  - Produces: Codebase map
- **Agent 2 - Literature Searcher**
  - Academic papers, official docs
  - Produces: Bibliography
- **Agent 3 - Pattern Identifier**
  - Design patterns, similar solutions
  - Produces: Precedent list



Token Allocation

- Per-agent: ∼2,000–3,000 tokens
- Total: ∼9,000 tokens (3 agents)
- Trade-off: +6,000 for better quality

Pattern 2: Data Processing & Experiments



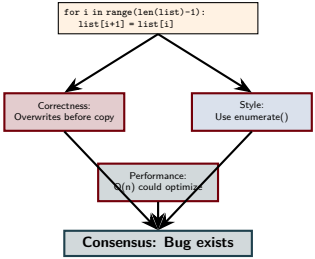
Speedup: 60–70% Faster

Data processing tasks see the highest improvements because they're most parallelizable with minimal synchronization points.

Pattern 3: Code Analysis & Debugging

Three-Agent Code Review

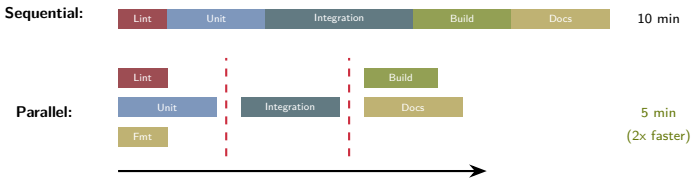
- **Correctness Reviewer**
  - Logical errors, edge cases
  - Off-by-one errors
- **Style Reviewer**
  - Readability, naming
  - Structure improvements
- **Performance Reviewer**
  - Efficiency, memory
  - Algorithmic improvements



Debugging Pattern

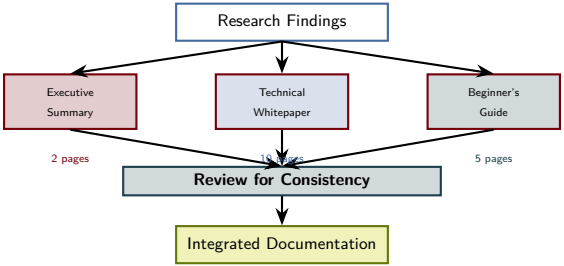
- Agent 1: Trace data flow
- Agent 2: Search for similar bugs
- Agent 3: Review related tests
- Agent 4: Integrate and fix

Pattern 4: Build & Test Parallelization



Phase 1	Phase 2	Phase 3
Lint, Unit Tests, Format (parallel)	Integration Tests (after Phase 1)	Build + Docs (parallel)

Pattern 5: Content Generation



Use Case

Generate different content pieces from the same source material in parallel. Each piece is independent—beginner's guide doesn't depend on technical whitepaper.

Explicit Parallel Request Syntax

Magic Keywords

- "in parallel"
- "simultaneously"
- "concurrently"
- "at the same time"

Basic Syntax

```
After completion, [synthesis]
```

Good vs Poor Requests

Poor (Ambiguous)

"Explore the code. Search for papers. Find patterns."

Good (Explicit)

"Run three agents in parallel:

1. Explore the codebase structure
2. Search for relevant papers
3. Find similar implementations

Then combine findings."

Maximum Concurrent Agents and Scaling

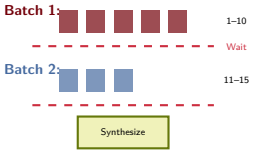
Maximum Parallel Agents: 10

Use Case	Recommended
Research	3–5 agents
Data processing	5–10 agents
Code analysis	3–4 agents
Build/testing	5–8 agents

Resource Implications

- 1 agent: ∼500MB memory
- 5 agents: ∼2.5GB memory
- 10 agents: ∼5GB memory

Beyond 10: Batching Strategy



Token Formula

N agents $\approx$ N $\times$ baseline tokens

The run_in_background Parameter

When to Use Background Execution

- Long data processing (> 10 minutes)
- Extensive experimentation
- Building large projects
- Running full test suites

When NOT to Use

- Tasks under 5 minutes
- Tasks needed for next step
- Interactive debugging

Pattern

Start long task in background → Do other work → Retrieve results → Synthesize

Ready-to-Use Prompt Templates

Template 1: Research

"Run 3 agents in parallel:

- Agent 1 - Codebase Explorer
- Agent 2 - Literature Searcher
- Agent 3 - Pattern Researcher

After all complete, synthesize into report."

Template 2: Code Review

"Review from 3 perspectives:

- Agent 1 - Correctness
- Agent 2 - Style
- Agent 3 - Performance

Consensus: Issues 2+ agree on?"

Template 3: Experiments

"Run experiments in parallel:

- Agent 1: param=value1
- Agent 2: param=value2
- Agent 3: param=value3

After all, compare and identify optimal."

Template 4: Documentation

"Generate docs in 3 formats:

- Agent 1: Executive summary
- Agent 2: Technical guide
- Agent 3: API reference

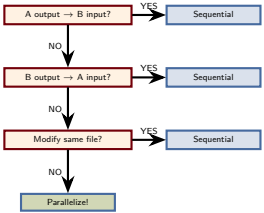
Combine into integrated package."

Independence Requirements

Independence Checklist

- ☐ Task A's output NOT required by Task B
- ☐ Task B's output NOT required by Task A
- ☐ Both don't modify same files
- ☐ Order of execution doesn't matter
- ☐ Results can be combined without conflicts

All TRUE → Can parallelize



Specificity for Each Agent

Good Role Naming

Vague

Agent 1, Agent 2, Agent 3

Specific

- Agent 1 - Correctness Reviewer
- Agent 2 - Performance Analyzer
- Agent 3 - Security Auditor

Clear Task Instructions

Vague

"Analyze this code"

Specific

"Review this code for security vulnerabilities. Look for SQL injection, buffer overflows, privilege escalation. Provide specific code locations and severity levels."

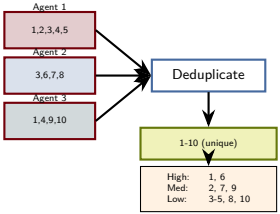
Output Specification

Always specify expected output format (e.g., JSON, structured list, specific metrics).

Collecting and Synthesizing Results

Synthesis Approaches

- **MERGE**: Combine into single output
- **CONSENSUS**: Look for agreement
- **COMPARE**: Highlight differences
- **RANK**: Order by importance



Handling Contradictions

- If Agent 1 says issue, Agent 2 says not
- Compare their reasoning
- Determine who is more likely correct

Token Usage Monitoring

Token Cost Model

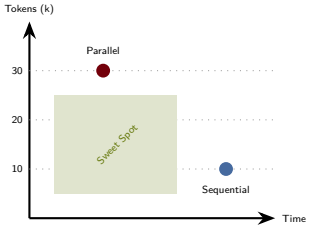
- Single agent: 3,000 tokens
- 3 parallel agents: ∼9,000 tokens
- Cost multiplier: 3x

Cost-Benefit Analysis

- Extra tokens: 6,000
- Time saved: 5 minutes
- Worth it if: time value > token cost

Optimization Techniques

- Context compression
- Focus narrowing
- Smaller models for parallel
- Result deduplication



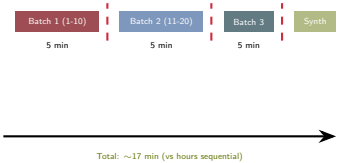
ROI Formula

ROI = Time Value - Token Cost
Positive = Worth parallelizing

Handling Large-Scale Parallelization

When You Need $>$ 10 Agents

- Large experiments (many parameters)
- Big data processing (multiple datasets)
- Comprehensive code analysis
- Extensive testing



Batching Strategy

- Need 25 agents? Use 3 batches
- Batch 1: Agents 1–10
- Batch 2: Agents 11–20
- Batch 3: Agents 21–25
- Final: Combine all results

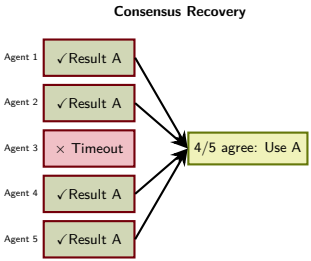
Resource Consideration

Running 10 concurrent agents is resource-intensive. Space agents across batches rather than cramming 20 into

Error Recovery and Fault Tolerance

Fault Tolerance Levels

- **Level 1 (NONE)**: 1 fails → task fails
- **Level 2 (PARTIAL)**: Use results from others
- **Level 3 (FULL)**: Retry or use backup



Redundancy Pattern

- N=1: No tolerance
- N=3: Can continue if 1 fails
- N=5: Can tolerate 2 failures

Fallback Strategies

- Retry parallel with timeout extension
- Run sequential version
- Use last known good results

Key Takeaways

1. Independence is Everything

Parallelize only truly independent tasks. If B needs A's output, must run sequentially.

2. Explicit is Better

Use clear "run in parallel" language. Give each agent specific role and instructions.

3. Cost-Benefit Analysis

3x tokens for 3 agents. Worth it when time saved $>$ token cost.

4. Synchronization Matters

Plan synthesis step carefully. Multiple perspectives improve quality through consensus.

5. Scale Pragmatically

3–5 agents usually optimal. Use batching for larger tasks (beyond 10).

6. Workflow Integration

Research: Heavy parallel. Implementation: Selective. Testing: Moderate. Refactoring: Sequential.

Common Pitfalls to Avoid

Parallelizing Dependent Tasks

× "Parse data in parallel with processing"
✓"Parse first, then process in parallel"

Vague Agent Instructions

× "Agent 1: Analyze code"
✓"Agent 1 - Security Auditor: Look for SQL injection..."

Ignoring Token Cost

× Always parallelize for speed
✓Parallelize if token cost $<$ time value

No Synthesis Plan

× Run 3 agents, no plan for combining
✓Plan synthesis step explicitly

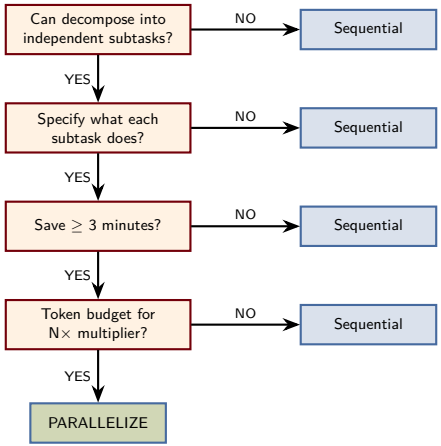
Expecting Perfect Consensus

× Assume identical results
✓Expect differences, look for patterns

Resource Exhaustion

× 20 agents on 4GB RAM
✓Monitor resources, use 3–5 agents

Decision Flowchart: Should I Parallelize?



Next Steps and Further Learning

Immediate Next Steps

1. Identify a parallelizable task
2. Use templates from this presentation
3. Measure results (time, tokens, quality)
4. Document what you learned

Short-Term Goals (2–3 weeks)

- Master 3-agent research pattern
- Master 3-agent code review pattern
- Use in at least 5 distinct tasks
- Build personal template library

Development Path

Questions to Explore

- What tasks parallelize best in your domain?
- Optimal team size for your work?
- How to measure quality improvements?

Appendix A: Quick Reference Templates

Three-Agent Research

```
Run 3 agents in parallel:
Agent 1 - Codebase Explorer:
  Explore [PROJECT] for [TOPIC].
Agent 2 - Literature Researcher:
  Search papers/docs for [TOPIC].
Agent 3 - Pattern Identifier:
  Find similar implementations.
After completion, synthesize.
```

Code Review Panel

```
Review from 3 angles:
Agent 1 - Correctness:
  Check logic, edge cases, errors.
Agent 2 - Style:
  Check readability, naming.
Agent 3 - Performance:
  Check efficiency, algorithms.
Consensus: Issues 2+ agree on?
```

Appendix B: Troubleshooting Guide

Problem	Cause	Fix
One agent times out	Unbalanced workload	Redistribute work evenly
Contradictory results	Different interpretation	More specific instructions
Synthesis fails	Format not specified	Specify exact output format
Quality worse than seq	Vague instructions	Give focused roles
Token cost too high	Too many agents	Reduce agent count
Parallelization slower	Overhead exceeds savings	Use sequential for small tasks
Empty agent output	Silent failure	Make requirements explicit

Appendix C: Metric Formulas

Performance Metrics

$$\text{Speedup} = \frac{\text{Seq\_Time}}{\text{Par\_Time}}$$

$$\text{Token\_Mult} = \frac{\text{Par\_Tokens}}{\text{Seq\_Tokens}}$$

$$\text{Cost/Min} = \frac{\Delta\text{Tokens}}{\Delta\text{Time}}$$

ROI Calculation

$$\text{Time\_Value} = \frac{\text{Min\_Saved}}{60} \times \text{Rate}$$

$$\text{Token\_Cost} = \frac{\text{Extra\_Tokens}}{10^6} \times \text{Cost}$$

$$\text{ROI} = \text{Time\_Value} - \text{Token\_Cost}$$

Parallelization Worth If:

- (Speedup > 1.3) AND (Token_Mult < 3)
- OR Quality_Improvement $> 20\%$

Questions?

Part 7: Parallelization with Agents

University of South Carolina
Graduate Research Seminar